

APRENDIZADO NÃO-SUPERVISIONADO

EAB067 - Aprendizado de Máquina

Atualizado em: 20 de março de 2026

Iago Carvalho

Departamento de Ciências da Computação

Programa de Pós-Graduação em
Estatística Aplicada e Biometria



Como vimos nas aulas anteriores, algoritmos de aprendizado de máquina não-supervisionados utilizam dados **não rotulados**

Além disso, também vimos que suas principais aplicações são

- Clusterização
- Associação
- Redução de dimensionalidade

Hoje vamos entender melhor as tarefas de clusterização e de associação

CLUSTERIZAÇÃO

Um algoritmo de **clusterização** tenta agrupar determinadas entradas em **grupos**

- Agrupamento definido por similaridade das variáveis de entrada
- Número de grupos pode ser fixo ou dinâmico

Existem diversos algoritmos de clusterização

Aqui nós vamos falar somente sobre o K-Means, mas vamos ver a implementação de diversos outros na próxima aula

O algoritmo K-Means está entre os métodos não-supervisionados mais utilizados

- Método não-paramétrico
- Utiliza *lazy learning*
- Útil para classificação e regressão

O objetivo do K-Means é de particionar um conjunto de dados em K conjuntos (clusters) sem sobreposição

- Ele minimiza a distância entre os pontos e K centróides definidos com base nos dados
- Minimiza a **variância intra-cluster**

A MATEMÁTICA POR TRÁS DO K-MEANS

O objetivo do K-Means é particionar um conjunto de n observações $X = \{x_1, x_2, \dots, x_n\} \in \mathbb{R}^d$ em K *clusters* $S = \{S_1, S_2, \dots, S_K\}$ como forma a minimizar a Soma de Quadrados Intra-Cluster (WCSS, do Inglês Within-Cluster Sum of Squares).

$$\arg \min_S \sum_{i=1}^K \sum_{x \in S_i} \|x - \mu_i\|^2$$

- K é o número de *clusters*
- S_i é o conjunto de pontos do i -ésimo *cluster*
- μ_i é o ponto médio (centróide) dos pontos em S_i

$$\mu_i = \frac{1}{|S_i|} \sum_{x \in S_i} x$$

ESCOLHA DOS CENTRÓIDES INICIAIS

Inicialmente escolhe-se um total de K pontos de maneira aleatória dentre todos os pontos disponíveis

- Escolha é equiprovável dentre todos os pontos existentes
- Complexidade de $\mathcal{O}(1)$

A ideia por trás disso é que regiões que possuem muitos pontos tendem a ser escolhidas como centróides

- Se tem muitos pontos, é porquê provavelmente é um *cluster*

RESOLUÇÃO DO K-MEANS

O K-Means é resolvido utilizando a **heurística de Lloyd**

- Complexidade de $\mathcal{O}(K \cdot n \cdot d)$

Este é um algoritmo iterativo composto de dois passos

1. Cada ponto é atribuído ao *cluster* com menor distância

$$S_i^{(t)} = \{x : \|x - \mu_i^{(t)}\|^2 \leq \|x - \mu_j^{(t)}\|^2, \quad \forall j \in 1, \dots, K\}$$

2. Novos centróides são calculados considerando a nova distribuição de *clusters*

$$\mu_i^{(t+1)} = \frac{1}{|S_i^{(t)}|} \sum_{x \in S_i^{(t)}} x$$

Estes passos são repetidos iterativamente até que $\mu_i^{(t+1)} \geq \mu_i^{(t)}$

O K-Means++ é um método mais inteligente utilizado para escolher os centróides iniciais do K-Means

1. O centróide μ_1 é escolhido aleatoriamente com probabilidade uniforme
2. Para todos os pontos restantes, computa-se a distância $D(x)$ do ponto até o centroide mais próximo

$$D(x) = \min \|x - \mu_i\|, \forall i$$

3. Escolhe-se um novo centróide probabilisticamente dentre os pontos que sobraram. A probabilidade $P(x)$ é proporcional a distância $D(x)^2$ de todo ponto, computada como

$$P(x) = \frac{D(x)^2}{\sum_{x \in X} D(x)^2}$$

Quando o conjunto de dados é extremamente grande e não cabe na memória, utilizamos o **Mini-Batch K-Means**

Nesta implementação, a heurística de Lloyd utiliza um conjunto $X' \subset X$ para calcular novos centróides

- Temos que $|X'| \lll |X|$
- O tempo computacional também diminuirá

O K-Means pode ser utilizado para realizar inferências quando acoplado a um algoritmo de regressão

- Linear, logística, quadrada, ...

Um modelo de regressão é treinado para cada *cluster*

- Quando um dado novo chega, verifica-se sua proximidade com todos os centróides
- Aplica-se o modelo de regressão do *cluster* que possui o centróide mais próximo

ESCOLHENDO O NÚMERO K DE CLUSTERS

Uma ótima abordagem é a análise de silhueta dos *clusters*

Calcula-se o valor da silhueta $s(x)$ de um ponto $x \in X$ como

$$s(x) = \frac{b(x) - a(x)}{\max\{a(x), b(x)\}},$$

onde

$$a(x) = \sum_{p \in S_i} \|x - p\|^2 \mid x \in S_i \quad (\text{Distancia intra-cluster})$$

e

$$b(x) = \sum_{p \in S_i} \|x - p\|^2, \forall S_i \in S \mid x \notin S_i \quad (\text{Distancia inter-cluster})$$

ESCOLHENDO O NÚMERO K DE CLUSTERS

O valor de silhueta s do K-Means é igual a média dos valores de silhueta de todos seus pontos

$$s = \frac{1}{n} \sum_{x \in X} s(x)$$

O valor de silhueta pode variar entre -1 e 1

- Quanto mais próximo de 1 , melhor é a clusterização

Este método é melhor do que o método do cotovelo pois leva em consideração a distância intra- e inter-cluster

- O método do cotovelo só analisa a distância intra-cluster

CUIDADOS AO USAR O K-MEANS

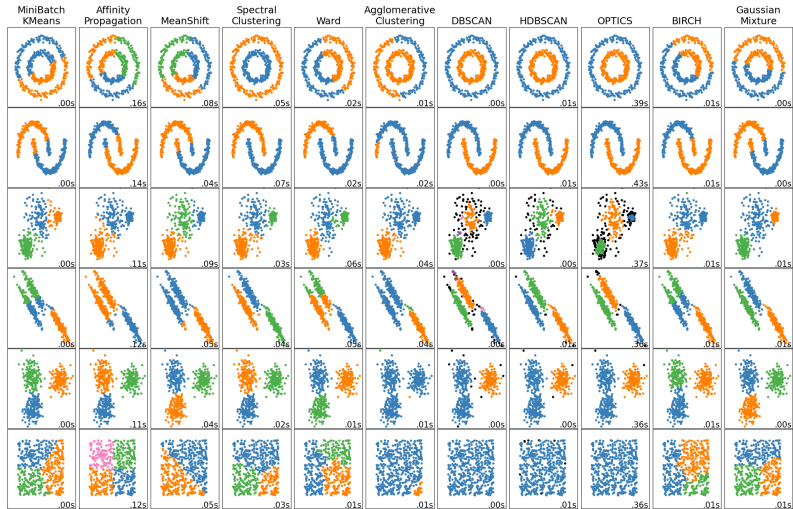
Dados devem ser normalizados

- A normalização evita que variáveis com *range* alto dominem a análise

Existem dois principais métodos de normalização

- **Min-Max Scaling**: faz com que todas as variáveis tenham valores no intervalo $[0, 1]$
- **Padronização de Z-scores**: traz a média das variáveis para zero e variância para 1

OUTROS ALGORITMOS PARA CLUSTERIZAÇÃO



ASSOCIAÇÃO

ASSOCIAÇÃO DE DADOS

Associações são co-ocorrências entre dados

- Útil em aplicações reais para descobrir o que *casa* com o quê
- Principais algoritmos: **Apriori**, Eclat, FPGrowth, AIS

Muito útil em sistemas de recomendação e sistemas médicos

- Se alguém assistiu o filme A, provavelmente também gostará do filme B
- Se o paciente tem sintomas X e Y, provavelmente ele tem uma doença Z

Algoritmos de associação funcionam bem com variáveis categóricas

- Pode ser necessário categorizar variáveis contínuas

Este é um ponto fundamental em **mineração de dados**

Uma regra de associação entre duas variáveis X e Y pode ser descrita como

$$\text{Antecedente (X)} \Rightarrow \text{Consequente (Y)}$$

Regra de associação

Se a variável X ocorre, então é provável que a variável Y também ocorra

APRENDIZADO DE REGRAS DE ASSOCIAÇÃO

Para que uma regra de associação seja considerada **forte**, ela deve satisfazer três parâmetros

- **Suporte:** Probabilidade que uma transação contenha ambos X e Y
 - Mede a utilidade da regra
 - Varia entre 0 e 1
- **Confiança:** Probabilidade condicional. Dado que X está presente, qual é a chance de que Y também esteja?
 - Mede a confiabilidade da regra
 - Varia entre 0 e 1
- **Lift:** A probabilidade de observar Y é diferente quando sabemos que X está presente?
 - Mede o desvio em relação a independência estatística de Y
 - Varia entre 0 e ∞

APRENDIZADO DE REGRAS DE ASSOCIAÇÃO - SUPORTE

O **suporte** s pode ser computado como

$$s(X \cup Y) = \frac{\text{Quantidade}(X \cup Y)}{\text{Transações totais}}$$

Mede qual é a probabilidade de X e Y aparecerem em conjunto

- Probabilidade conjunta de X e Y

Um bom valor de suporte depende da aplicação

- Envolvendo duas variáveis, um suporte de 0,2 é alto
- Caso o suporte envolva múltiplos itens, pode-se considerar como altos valores a partir de 0,05
- Em comércio eletrônico, um suporte maior que 0,01 já é relevante

APRENDIZADO DE REGRAS DE ASSOCIAÇÃO - CONFIANÇA

A confiança c pode ser computado como

$$c(X \Rightarrow Y) = \frac{\text{Quantidade}(X \cup Y)}{\text{Quantidade}(X)} = \frac{s(X \cup Y)}{s(X)}$$

A confiança mede a força condicional da dependência

- Dado que uma transação já possui o antecedente (X), qual é a probabilidade que ela também possua o conseqüente (Y)?
- $c(X \Rightarrow Y) \approx P(\text{Item}(s) \text{ em } Y \text{ estão presentes} \mid \text{Item}(s) \text{ em } X \text{ estão presentes})$

A confiança é assimétrica, isto é

$$c(X \Rightarrow Y) \neq c(Y \Rightarrow X)$$

Exemplo: Caviar e Champagne

Um valor alto de confiança ($c > 0,8$) é **necessário**, mas não é **suficiente** para declarar uma regra de associação forte

- Este valor deve ser confirmado utilizando o *lift*

Imagine o seguinte exemplo:

$$c(\text{banana} \Rightarrow \text{sacola}) = 0,99 \text{ e } c(\text{carne} \Rightarrow \text{sacola}) = 0,99$$

Neste caso, não temos nenhuma associação. Só sabemos que praticamente todo mundo compra sacola

APRENDIZADO DE REGRAS DE ASSOCIAÇÃO - *LIFT*

O *lift* (l) pode ser computado como

$$l(X \Rightarrow Y) = \frac{c(X \Rightarrow Y)}{s(Y)} = \frac{s(X \cup Y)}{s(X) \cdot s(Y)}$$

Ele mede o relacionamento, correlação ou dependência entre o antecedente (X) e o consequente (Y)

- Determina se a associação entre X e Y é genuína ou só uma coincidência

Pode-se observar que $s(X) \cdot s(Y)$ é igual a probabilidade conjunta de dois eventos independentes X e Y

$$s(X) \cdot s(Y) = P(X \cap Y) = P(X) \cdot P(Y)$$

Assim, temos o desvio da independência dos eventos

APRENDIZADO DE REGRAS DE ASSOCIAÇÃO - *LIFT*

Podemos obter diferentes conclusões a partir dos valores de *lift*

Valor	Definição matemática	Significado estatístico
$I = 1$	$P(X \cap Y) = P(X) \cdot P(Y)$	Independência
$I > 1$	$P(X \cap Y) > P(X) \cdot P(Y)$	Dependentes
$I < 1$	$P(X \cap Y) < P(X) \cdot P(Y)$	Dependentes

Se os conjuntos forem independentes, então não existe nenhuma regra de associação entre X e Y

Se os conjuntos forem dependentes, devemos analisar a correlação

- Correlação positiva ($I > 1$): X e Y se complementam; eles ocorrem em conjunto com probabilidade maior que a esperada
- Correlação negativa ($I < 1$): X e Y se repelem ou se substituem; eles ocorrem em conjunto com probabilidade menor que a esperada

APRENDIZADO DE REGRAS DE ASSOCIAÇÃO - APRIORI

A maior dificuldade em encontrar regras de associação é o tamanho do espaço de buscas

- Se existem T diferentes itens ou variáveis, então existem $2^T - 1$ diferentes conjuntos únicos

O algoritmo Apriori é extremamente utilizado para aprender regras de associação em grandes bancos de dados

Ele diminui o espaço exponencial de itens utilizando duas propriedades relativas ao suporte s das regras de associação

- Propriedade de fechamento descendente
 - *Downward Closure Property*
- Propriedade anti-monotônica do suporte
 - *Anti-Monotone Property of Support*

Propriedade de fechamento descendente: Se um conjunto T é frequente, então todos os subconjuntos de T também devem ser frequentes

$$s(T) \geq \varepsilon \implies s(S) \geq \varepsilon, \quad \forall S \subseteq T$$

Propriedade anti-monotônica do suporte: De forma contrária, temos que se um conjunto T é infrequente, então todos os superconjuntos de T também são infrequentes

$$s(T) < \varepsilon \implies s(Q) < \varepsilon \quad \forall Q \supseteq T$$

A propriedade anti-monotônica do suporte é utilizada pelo Apriori para podar o espaço de buscas

O algoritmo Apriori equivale a um algoritmo de busca em largura utilizando métodos de poda

- Ele itera sobre o conjunto de dados k vezes para gerar k -conjuntos (chamados de L_k)

Ele recebe duas entradas

- O conjunto de transações D
- Um valor de *threshold* $\varepsilon \in [0, 1]$

APRENDIZADO DE REGRAS DE ASSOCIAÇÃO - APRIORI

Primeira iteração: Ele encontra todos os 1-conjuntos (L_1)

- O algoritmo percorre o conjunto D em busca de todas as transações de um único item
- Se a frequência do item é menor do que ϵ , então ele é descartado
- Se o item não for descartado, então ele forma um L_1

Iterações subsequentes: Até que nenhum conjunto frequente de itens seja gerado, o algoritmo utiliza o conjunto L_{k-1} para gerar L_k

Esse processo é dividido em 4 passos

1. Geração de candidatos
2. Poda de candidatos
3. Cálculo do suporte
4. Definição dos *supersets*

Geração dos candidatos: Para minimizar a quantidade de candidatos redundantes, dois conjuntos de L_{k-1} são unidos e inseridos em L_k se seus primeiros $k - 2$ itens são iguais e seus últimos elementos são diferentes.

Exemplo: Se $L_2 = \{\{A, B\}, \{A, C\}, \{B, C\}\}$, nós juntamos $\{A, B\}$ e $\{A, C\}$ (mesmo primeiro elemento) em um candidato potencial $\{A, B, C\}$.

Poda de candidatos: Para todo candidato c_k gerado na etapa anterior, o algoritmo verifica se L_{k-1} contém todos os subconjuntos próprios de c_k com exatamente $k - 1$ itens

- Se qualquer subconjunto próprio não está L_{k-1} , então c_k não pode ser frequente (baseado na propriedade anti-monotônica)
- Deste modo, c_k é removido de L_k

Exemplo: Para o candidato $\{A, B, C\} \in L_3$, seus subconjuntos próprios avaliados são $\{A, B\}$, $\{A, C\}$ e $\{B, C\}$. Se $\{A, C\}$ não estiver em L_2 , então $\{A, B, C\}$ é removido de L_3 .

Cálculo do suporte: O algoritmo calcula o suporte de todos os candidatos que sobraram no conjunto L_k

- Se $L_k = \emptyset$, então o algoritmo para

Definição dos supersets: Se L_k não for vazio

- Ele calcula a frequência de todos os conjuntos de itens em L_k
- Se a frequência for menor que ε , então o conjunto é removido de L_k
- Se $L_k = \emptyset$, o algoritmo para
 - Caso contrário, ele tenta computar L_{k+1}

Após computar o maior L_k possível, é necessário extrair as regras de associação

O objetivo é verificar todos os conjuntos $T \in L_k$ e particiona-los em dois conjuntos não-vazios e sem sobreposição

- Conjunto de antecedentes (X) e de consequentes (Y)
- $X \cap Y = \emptyset$
- $X \cup Y = T$

A regra associativa candidata $X \Rightarrow Y$ é considerada forte se sua confiança (c) é maior que uma confiança mínima γ estabelecida

Primeiro passo: particionamento iterativo de itens (se $|T| \geq 2$)

1. Gerar todos os subconjuntos próprios de T
 - Estes serão os antecedentes X
2. Gerar o complemento de todos os subconjuntos próprios de T
 - Estes serão os consequentes Y

Exemplo do primeiro passo: Seja $T = \{\text{leite}, \text{pão}, \text{ovo}\}$ um conjunto de L_3

Temos as seguintes regras de associação candidatas

1. $X = \{\text{leite}\}, Y = \{\text{pão}, \text{ovo}\} \implies \{\text{leite}\} \Rightarrow \{\text{pão}, \text{ovo}\}$
2. $X = \{\text{pão}\}, Y = \{\text{leite}, \text{ovo}\} \implies \{\text{pão}\} \Rightarrow \{\text{leite}, \text{ovo}\}$
3. $X = \{\text{ovo}\}, Y = \{\text{leite}, \text{pão}\} \implies \{\text{ovo}\} \Rightarrow \{\text{leite}, \text{pão}\}$
4. $X = \{\text{leite}, \text{pão}\}, Y = \{\text{ovo}\} \implies \{\text{leite}, \text{pão}\} \Rightarrow \{\text{ovo}\}$
5. $X = \{\text{leite}, \text{ovo}\}, Y = \{\text{pão}\} \implies \{\text{leite}, \text{ovo}\} \Rightarrow \{\text{pão}\}$
6. $X = \{\text{pão}, \text{ovo}\}, Y = \{\text{leite}\} \implies \{\text{pão}, \text{ovo}\} \Rightarrow \{\text{leite}\}$

Segundo passo: aplicando a propriedade de anti-monotonicidade da confiança

- Assim como o suporte é anti-monotônico em relação ao tamanho do conjunto de itens, a confiança é anti-monotônica em relação ao tamanho do consequente da regra

Propriedade de anti-monotonicidade da confiança

Se a regra $X \Rightarrow (Y - y)$ tem baixa confiança, então a regra $(X - x) \Rightarrow Y$ também deve ter baixa confiança. Assim, temos que $c(X \Rightarrow Y) \leq c(X' \Rightarrow Y')$ onde $X \subseteq X'$ e $Y' \subseteq Y$

Exemplo do segundo passo: Seja $T = \{\text{leite, pão, ovo, batata}\}$ um conjunto de L_4

Se a regra $\{\text{leite, pão, ovo}\} \Rightarrow \{\text{batata}\}$ tem baixa confiança, então garantidamente terão baixa confiança as regras

- $\{\text{leite, pão}\} \Rightarrow \{\text{ovo, batata}\}$
- $\{\text{leite}\} \Rightarrow \{\text{pão, ovo, batata}\}$

Regras com confiança $c < \gamma$ podem ser descartadas

Terceiro passo: computando o *lift* das regras de associação candidatas

- O valor de *lift* l é computado para todas as regras de associação candidatas que possuem confiança $c \geq \gamma$
- Todas as regras com $l \leq 1$ são descartadas
- O que sobrar, são as regras de associação finais do algoritmo Apriori

Este terceiro passo envolve ler, novamente, todo o conjunto de transações D para computar o suporte das regras candidatas

ALGORITMO APRIORI - ESCOLHENDO O VALOR DE γ

A escolha de γ é uma heurística e pode impactar diretamente na qualidade dos *insights* gerados pelo algoritmo Apriori

Se $\gamma \geq 0,8$, então poucas regras de associação são criadas

- Estas regras serão extremamente fortes, mas podem representar relações muito óbvias
- Muito útil também para fazer a detecção de anomalias e verificar erros em sistemas de computação ou engenharia

Se $\gamma \leq 0,4$, um grande número de regras de associação são criadas

- Regras mais fracas e correm o risco de serem espúrias
- Úteis para fazer análise exploratória
 - Por exemplo, sugestão de compra de itens de nicho

1. **Alto número de buscas:** O número de vezes que é necessário ler completamente o banco de dados é igual ao valor de k . Se o banco D for grande e salvo em disco (ou na nuvem), então vai existir **muito** tráfego de dados
2. **Grande número de candidatos iniciais:** O conjunto L_1 pode ser extremamente grande, gerando *overhead* de processamento. Isto é prejudicial para os níveis baixos do algoritmo ($k = 2$ ou $k = 3$)
3. **Uso de memória:** O algoritmo Apriori assume que D caiba na RAM ou que grandes estruturas de candidatos caibam na RAM